An Empirical Evaluation of Source Code Chunking and Context Headers for Requirement Evidence Retrieval

Anonymous Authors

***Abstract*—Locating source code that evidences a technical requirement is fundamental in software analysis. While syntax-aware chunking is widely advocated to preserve code structure, prior evaluations conflate chunk boundaries with the contextual metadata prepended to chunks. In this Early Research Achievement (ERA) paper, we present a controlled $\times 2$ factorial evaluation crossing chunking strategy (node-level abstract syntax trees vs. 50-line windows) with context headers (present vs. absent). Evaluating across 25 job descriptions, 40 repositories, and three retrieval tiers (BM25, dense, cross-encoder reranking) against 882 human-labeled pairs, our findings show: (1) Context headers substantially boost AST chunks (NDCG@10 increases from 0.341 to 0.487, < 0.001**$),whiletheirgainforlinewindowsisminor(\Delta = +0.041, p = 0.2411)$**. (2) Simple 50-line windows match or exceed AST chunking across all comparable tiers; even headerless line windows significantly outperform AST chunks with headers** $(\Delta = -0.093, p = 0.0438)$**. (3) Line windows with headers achieve the highest overall retrieval effectiveness (0.621 vs. 0.487, = 0.0003**$).Theseresultsindicatethatcontextualdensityfrequentlyout$ [illegible] $cludewithan$

***Index Terms*—Source code retrieval, code chunking, abstract syntax trees, context headers, requirements traceability, empirical software engineering**

I. INTRODUCTION

Locating source code that provides concrete evidence for a technical requirement is fundamental in software analysis and capability assessment. While classical requirements traceability links formal specifications to code artifacts [1], [2], modern software engineering frequently demands verifying whether a codebase implements higher-level framework capabilities (e.g., reactive state management or token authentication). We formulate this task as **requirement and technical-capability evidence retrieval**: given natural-language technical requirements—instantiated via mandatory skill specifications from job descriptions (JDs)—the objective is to retrieve the specific code chunks that directly evidence their concrete implementation across multi-file repositories, without requiring explicit requirement IDs.

In retrieval-augmented generation (RAG) pipelines for software engineering, source code must be segmented into chunks for neural encoding. Chunking via abstract syntax trees (ASTs) is widely advocated to preserve semantic units (methods, declarations) [3]. However, recent empirical studies on code completion indicate that structure-aware chunking does not reliably outperform simple sliding windows [4], and line-based segmentations match syntax-aware splitting across compute budgets [5]. Crucially, prior work conflates chunk boundaries with the contextual metadata (file path, enclosing class, method signature) prepended to chunks. Whether structural chunking provides distinct advantages once metadata is controlled remains unexamined for requirement evidence retrieval.

In this Early Research Achievement (ERA) paper, we present a controlled 2×2 factorial evaluation crossing chunking strategy (node-level AST vs. 50-line windows) with context headers (present vs. absent). We address four research questions: (RQ1) does prepending a context header improve evidence retrieval? (RQ2) does node-level AST chunking outperform line windows? (RQ3, Roadmap) which header components drive retrieval gains across languages? and (RQ4, Roadmap) does retrieved evidence improve practitioner verification?

We evaluate RQ1 and RQ2 across 25 JDs and 40 open-source repositories (20 Java Spring Boot, 20 React/TypeScript; 434 line-window and 438 AST chunks) at three tiers: BM25, dense bi-encoder (BGE-M3), and cross-encoder reranking (BGE-Reranker-Base), evaluated against 882 blinded human-labeled pairs (weighted Cohen's $\kappa_w = 0.9822$). Findings reveal: (1) headers significantly improve AST chunks ($\Delta = +0.146$, $p_{\text{Holm}} < 0.001$), whereas the gain for line windows is not significant ($\Delta = +0.041$, $p_{\text{Holm}} = 0.2411$); (2) without headers, line windows significantly outperform AST chunks ($\Delta = -0.239$, $p_{\text{Holm}} < 0.001$); (3) headerless line windows significantly outperform AST chunks with headers ($\Delta = -0.093$, $p_{\text{Holm}} = 0.0438$); (4) line windows with headers achieve highest overall effectiveness (NDCG@10 = 0.621), outperforming AST with headers ($\Delta = -0.134$, unadjusted $p = 0.0003$); and (5) with the final parser configuration, the AST fallback rate is 0.0%, showing that the performance gap is not caused by fallback degeneration.

Contributions: (1) *Factorial Framework:* Isolating chunk boundaries from context headers in code retrieval. (2) *Empirical Findings:* Evidence that sliding windows match or exceed AST chunking, with headers substantially improving AST chunks (mechanism untested; Section V). (3) *Curated Benchmark:* Replication package with 882 dual-annotated pairs ($\kappa_w = 0.9822$; top-5 fully judged by construction). (4) *ERA Roadmap:* Actionable research agenda on header components, dependency graphs, and human verification.

II. RELATED WORK

Code Chunking for RAG: Line-based chunking can disrupt semantic boundaries [3]. cAST [3] addresses this by recursively partitioning AST nodes and merging siblings under size limits, reporting gains on code-completion and issue-resolution benchmarks. However, subsequent controlled experiments in code completion show that structure-aware chunking does not reliably outperform sliding windows [4], and line-based slicing matches syntax-aware splitting across compute budgets [5]. While those studies target code completion, we evaluate human-labeled requirement-to-code evidence retrieval and orthogonally control context headers.

Dense Retrieval and Reranking: Neural code search embeds natural-language queries and source code into shared representations [6], [7] (benchmarked on CodeSearchNet [8]). Modern dense models like BGE-M3 [9] enable multilingual semantic matching; two-stage pipelines integrate cross-encoder rerankers that jointly attend across query and chunk tokens [10], [11] to overcome single-vector limits. We adopt this architecture to examine interactions between chunk geometry, headers, and cross-encoders.

Context Enrichment: Isolated code chunks frequently lack enclosing class and package context. Prior approaches enrich completion prompts with repository context iteratively (RepoCoder [12]) or benchmark cross-file dependencies (RepoBench [13], CrossCodeEval [14]). We investigate a lightweight alternative: prepending standardized context headers to individual chunks prior to retrieval.

Requirements Traceability and Skill Mining: Tracing natural-language requirements to code has deep roots in software engineering [1], [2]. Separately, repository mining classifies developer expertise from framework usage [15]. We adopt the traceability perspective: treating required technical skills as requirements and retrieving concrete code evidence, without assessing individual developers.

III. APPROACH AND STUDY DESIGN

To separate chunk boundaries from contextual metadata, we employ a 2×2 factorial design evaluated across three retrieval tiers, governed by a frozen analysis protocol.

A. Corpus, Candidate Space, and Curation

Our corpus comprises 40 open-source GitHub repositories (20 Java Spring Boot, 20 React/TypeScript) spanning 189 source files (`sampling_frame_log.json`) with production patterns (controllers, services, DTOs, custom hooks, Redux stores, UI components). To focus on authentic evidence, curation targeted core architectural modules (mean 4.7 files/repo) via directory conventions prior to query extraction and without access to retrieval results. Licenses encompass 19 MIT (47.5%), 16 with no explicit license (GitHub default copyright, 40.0%), 4 Apache-2.0 (10.0%), and 1 AGPL-3.0 (2.5%); all source text is redistributed for non-commercial research replication and will be removed upon request by rights holders. Queries represent 25 job descriptions (15 Java, 10 React/TS) extracted via a frozen prompt on `gemini-2.5-flash` (run on 2026-10-09; `queries_frozen.json`). Each query combines title, domain, and 5–8 mandatory skills. Retrieval evaluates across all 40 repositories without language filtering: 434 line chunks and 438 AST chunks (872 instances across strategies).

B. Factorial Conditions and Retrieval Pipeline

We evaluate two orthogonal dimensions: **Chunking Strategy:** *Line-window chunking* slices files into windows of 50 lines of code (LOC) with 10-LOC overlap (step 40 LOC, minimum 6 LOC; 434 chunks, mean 41.0 LOC, median 50 LOC, 100% line coverage across all 189 files, 28.6% exceeding 512 tokens, 56.0% cutting syntactic boundaries). *AST chunking* extracts node-level units using Tree-sitter[1] parsers expanded to modern constructs: 438 chunks (134 methods/constructors, 96 arrow functions, 86 component wrappers, 62 functions, 28 class/interface/record/enum declarations, and 32 type/constant/config declarations; mean 26.7 LOC, median 16 LOC, 15.8% exceeding 512 tokens). AST chunks can be nested (e.g., a class declaration and its methods), so per-chunk LOC sums exceed the number of covered lines. Due to $6 \leq \text{LOC} \leq 85$ filtering, AST chunks cover 69.0% of lines (10,585/15,351) across 183/189 files (6 files contain zero AST chunks), 87.2% of AST chunks align with syntactic boundaries under our interval check (the remaining 56 are module-wrapped functions counted as cut despite being intact AST nodes), versus 44.0% of line windows. No AST chunks required line-based fallback under the final parser configuration (0.0%; 0/438 chunks). The six files lacking extractable units (6/189) contribute no AST chunks; the common-support analysis (Section IV-C) addresses this coverage difference. **Context Header Condition:** *Without Header* contains raw code. For *With Header*, line chunks prepend File and Lines cues (100%); AST chunks prepend File, enclosing Class (when nested, 35.6%), Method (node name: method, function, component, type), and Lines cues (`// File: ... [// Class: ...] // Method: ... // Lines: ...`). **Retrieval Tiers:** (1) *BM25* [16] (Rank-BM25 `BM25Okapi`, identifier regex tokenization). (2) *Dense* BAAI/bge-m3 [9] (1024d, 8192 tokens, cosine similarity). (3) *Reranking* BAAI/bge-reranker-base [17] (cross-encoder rescoring top-20 dense candidates, 512-token limit). Parsers use official Tree-sitter grammars.

C. Unified Blinded Pooling Protocol

To reduce pooling bias, we executed a single-wave Cranfield/TREC pooling protocol: top-5 candidates from all 12 systems (BM25, Dense, Rerank × 4 conditions) were pooled, yielding 1,500 candidate slots deduplicated to 882 unique pairs. Two software engineering (SE) annotators (the authors) independently labeled all 882 pairs after calibration on a pre-specified 3-point relevance rubric: **0** (irrelevant/boilerplate: imports, standard configuration, or code lacking implementation of the target capability), **1** (partially relevant/indirect: interface signatures, type definitions, or peripheral helpers), and **2** (direct evidence: concrete, executable implementation directly demonstrating at least one listed skill). Labels attach to (query, raw code chunk) coordinates, so with-header and no-header conditions share identical relevance ground truth. Annotators were blinded to system, header condition, rank, and score (chunk geometry cannot be concealed, making strategy blinding partial). Inter-annotator agreement reached almost perfect concordance: quadratic-weighted Cohen's $\kappa = 0.9822$ (linear $\kappa = 0.9764$, unweighted $\kappa = 0.9708$, 98.07% exact agreement). The 17 disagreements were resolved by a Lead Adjudicator using standardized rubrics; all pooled pairs carry final consensus labels (zero orphaned pairs, top-5 fully judged by construction).

---

[1] https://tree-sitter.github.io/

D. Evaluation Metrics and Hypothesis Testing

The primary metric is normalized discounted cumulative gain at rank 10 (NDCG@10) [18] (linear gain $\mathrm{rel}_i / \log_2(i+1)$). Unjudged items receive relevance 0; we also report binary preference (`bpref`) [19], NDCG@1, mean reciprocal rank (MRR@10), and Judged@10. Judged@k is the fraction of top-k retrieved chunks carrying a human label. The pre-specified confirmatory family (fixed in an analysis plan before the final evaluation run and included in the replication package) tests four paired comparisons over $N = 25$ queries at the rerank tier: **Comp 1** (AST+Header vs. Line No-Header), **Comp 2** (AST No-Header vs. Line No-Header), **Comp 3** (AST+Header vs. AST No-Header), and **Comp 4** (Line+Header vs. Line No-Header) via two-sided Wilcoxon signed-rank tests [20] with Holm-Bonferroni correction [21] ($\alpha = 0.05$). We report paired effect sizes Δ (mean per-query NDCG@10 difference $A - B$, with A the first-named condition, so negative values favor B) with 95% bootstrap CIs (10,000 resamples), plus exploratory contrast **Comp 5** (AST+Header vs. Line+Header). W denotes the Wilcoxon signed-rank statistic; "+H" and "No-H" abbreviate with and without header. Wilcoxon p-values are exact where no ties occur and use the normal approximation otherwise.

IV. EMPIRICAL RESULTS

A. Overall Benchmark Results across Twelve Systems

Table I presents the 12-configuration benchmark across lexical (BM25), dense bi-encoder (BGE-M3), and cross-encoder reranking tiers. At reranking, Line+Header achieves the highest NDCG@10 (0.621, 95% CI $[0.555, 0.682]$). It ties Line No-Header on NDCG@1 (0.800), while Line No-Header has a marginally higher MRR@10 (0.953 vs. 0.950). AST+Header reaches 0.487 ($[0.419, 0.551]$), and AST No-Header drops to 0.341 ($[0.263, 0.421]$).

TABLE I

RETRIEVAL EFFECTIVENESS ACROSS 12 CONFIGURATIONS ($N = 25$ JDS).

Tier	Chunk	Header	NDCG@10 [95% CI]	NDCG@1	MRR@10	bpref	J@10
Two-Stage Re-ranking Tier (Cross-Encoder)							
Rerank	Line	Yes	**0.621** [0.555, 0.682]	**0.800**	0.950	**0.353**	0.736
Rerank	Line	No	*0.580* [0.505, 0.649]	**0.800**	**0.953**	*0.300*	0.716
Rerank	AST	Yes	0.487 [0.419, 0.551]	0.740	0.896	0.241	0.660
Rerank	AST	No	0.341 [0.263, 0.421]	0.340	0.699	0.170	0.712
Dense Bi-Encoder Tier (BGE-M3)							
Dense	Line	Yes	**0.551** [0.480, 0.619]	0.580	0.883	**0.347**	0.752
Dense	Line	No	0.521 [0.439, 0.600]	0.600	0.891	0.297	0.720
Dense	AST	Yes	0.450 [0.393, 0.506]	**0.620**	**0.910**	0.247	0.692
Dense	AST	No	0.358 [0.298, 0.420]	0.420	0.813	0.179	0.716
Lexical Sparse Baseline Tier (BM25)							
BM25	Line	Yes	**0.433** [0.376, 0.486]	**0.440**	**0.868**	**0.255**	0.696
BM25	Line	No	0.405 [0.350, 0.458]	**0.440**	0.865	0.246	0.644
BM25	AST	Yes	0.311 [0.243, 0.376]	0.340	0.761	0.137	0.676
BM25	AST	No	0.305 [0.236, 0.371]	0.340	0.724	0.129	0.560

Bold: best within tier; italic: second best (Rerank tier). Linear gain $\mathrm{rel}/\log_2(i+1)$. J@10: Judged@10 coverage.

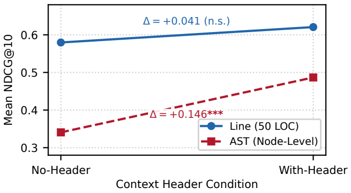

Fig. 1. Interaction plot of context header sensitivity at reranking ($N = 25$).

Dense encoding improves NDCG@10 over BM25 by 0.118–0.139 with headers, and reranking adds another 0.037–0.070. Header gains differ across chunking strategies. For AST chunks, they grow with retriever capacity: negligible for BM25 (+0.006), moderate for dense retrieval (+0.092), and large for reranking (+0.146). For line windows, gains are modest at every stage (+0.028, +0.030, +0.041). Binary preference (`bpref`) orders the four conditions identically to NDCG@10 at every tier (Table I). At the reranking tier: Line+H 0.353 > Line No-H 0.300 > AST+H 0.241 > AST No-H 0.170. The bpref results preserve the same condition ordering, though unjudged items at ranks 6–10 cannot be ruled out. Two patterns deserve mention: at the dense tier AST+Header attains higher NDCG@1 (0.620 vs. 0.580) and MRR@10 (0.910 vs. 0.883) than Line+Header, although it trails on NDCG@10 (0.450 vs. 0.551); reranking improves every condition over dense except AST No-Header, dropping from 0.358 to 0.341, consistent with cross-encoder reliance on header cues for isolated AST chunks.

B. Hypothesis Testing and RQ Analyses

Table II reports the pre-specified confirmatory Wilcoxon signed-rank tests with Holm-Bonferroni correction, alongside the exploratory contrast.

RQ1 (Context Headers): Prepending headers yields a large, significant gain for AST chunks (**Comp 3**: $\Delta = +0.146$, 95% CI $[+0.086, +0.207]$, $p_{\mathrm{Holm}} < 0.001$, $W = 24.0$; AST+H wins 23/25 JDs). In contrast, prepending headers to line windows yields a minor, non-significant gain (**Comp 4**: $\Delta = +0.041$, 95% CI $[-0.012, +0.093]$, $p_{\mathrm{Holm}} = 0.2411$, $W = 118.0$; 15 wins vs. 10 losses). A post hoc exploratory difference-in-differences test (unadjusted) indicates that the header effect differs between strategies ($W = 66.0, p = 0.0081$). As shown in Fig. 1, headers significantly improve AST chunks, whereas the gain for 50-line windows is not statistically significant.

RQ2 (AST vs. Line): Without headers, RQ2 is tested by Comp 2 (confirmatory); with headers, by Comp 5 (exploratory). Comp 1 additionally compares across header conditions (AST+Header vs. Line No-Header). Without headers, line windows strongly outperform AST chunks (**Comp 2**: $\Delta = -0.239$, 95% CI $[-0.292, -0.185]$, $p_{\mathrm{Holm}} < 0.001$, $W = 3.0$; Line wins 23/25). Line windows without headers also significantly outperform AST chunks with headers (**Comp 1**: $\Delta = -0.093$, 95% CI $[-0.164, -0.023]$, $p_{\mathrm{Holm}} = 0.0438$, $W = 78.0$; Line

TABLE II

HYPOTHESIS TESTS ON NDCG@10 AT RERANKING TIER ($N = 25$).

ID	Contrast ($A - B$)	Δ [95% CI]	W/L/T	p_{Holm}
Confirmatory family (step-down Holm, $\alpha = 0.05$)				
Comp 1	AST+Header vs. Line No-Header	-0.093 $[-0.164, -0.023]$	7/18/0	0.0438*
Comp 2	AST No-Header vs. Line No-Header	-0.239 $[-0.292, -0.185]$	2/23/0	< 0.001***
Comp 3	AST+Header vs. AST No-Header	$+0.146$ $[+0.086, +0.207]$	23/2/0	< 0.001***
Comp 4	Line+Header vs. Line No-Header	$+0.041$ $[-0.012, +0.093]$	15/10/0	0.2411
Exploratory (outside the Holm family)				
Comp 5	AST+Header vs. Line+Header	-0.134 $[-0.199, -0.070]$	5/20/0	$0.0003^{\dagger}$

W/L/T: queries where A wins, loses, or ties against B. Significance follows p_{Holm} (* < 0.05, ** < 0.01, *** < 0.001). †Unadjusted p (exploratory).

wins 18 vs. AST wins 7). In the exploratory contrast with headers (**Comp 5**), line windows remain superior ($\Delta = -0.134$, 95% CI $[-0.199, -0.070]$, unadjusted $p = 0.0003$, $W = 36.0$; Line wins 20 vs. AST wins 5).

C. Subgroups, Morphology Sensitivity, and Header Effects

Subgroups & Common Support: The direction is consistent in both subgroups (Java $N = 15$, React/TS $N = 10$; no per-language tests are claimed): Java yields Line+H 0.716 vs. AST+H 0.583 ($\Delta = -0.133$); React/TS yields 0.478 vs. 0.342 ($\Delta = -0.136$). Evaluating on common support (826/882 pairs covering 91.8% of relevant evidence across all 25 queries), AST+H remains significantly below Line+H (0.489 vs. 0.600, $\Delta = -0.111$, $p_{\mathrm{Holm}} = 0.0036$, $W = 51.0$), and AST No-H trails Line No-H (0.342 vs. 0.535, $\Delta = -0.193$, $p_{\mathrm{Holm}} < 0.001$, $W = 11.0$; Holm-adjusted). Restricting to common support removes 17.3% of the end-to-end gap (shrinking by 0.023, $p = 0.0015$); the lead persists, suggesting that coverage differences alone do not account for the observed gap.

Window Overlap Sensitivity: Suppressing overlapping windows from the same file in the top-10 (26.4% of Line top-10 slots involved in overlap [16.8% redundant], vs. 0% in AST No-Header and 0.8% in AST+Header, the latter due to nested units; Section III-B) lowers Line NDCG@10 slightly (Line+H 0.621 to 0.583, Line No-H 0.580 to 0.574) but preserves the ranking of conditions: Line+H 0.583, Line No-H 0.574, AST+H 0.482, AST No-H 0.341. Line windows retain their lead (**Comp 5**: $\Delta = -0.101$, unadjusted $p = 0.0013$; **Comp 1**: $\Delta = -0.092$, unadjusted $p = 0.0236$), and the AST header boost remains large (**Comp 3**: $\Delta = +0.141$, unadjusted $p < 0.001$).

Qualitative Inspection of Header Lift: Inspecting rank trajectories illustrates how headers ground isolated chunks. For `JD_JAVA_01` (*"Spring Security & JWT"*; JWT = JSON Web Token), a chunk in `AccountController.java` (lines 39–64, `me()`, AST arm) that retrieves authenticated user context via `userService.getUser()` rises from Rank 6 without header to Rank 1 with header (human relevance 2). For `JD_REACT_01` (*"React Admin Dashboard & RBAC"*; RBAC = role-based access control), an administrative control panel in `Admin/index.tsx` (lines 544–613, AST arm) jumps from Rank 13 to Rank 1 with header (relevance 2).

V. DISCUSSION AND THREATS TO VALIDITY

A. Underlying Mechanisms and Scope

In our setting, node-level AST extraction (without sibling merging as in cAST [3]) did not surpass sliding windows in requirement evidence retrieval. Three mechanisms may explain this; we did not test them directly: (1) *Information density and token budget*: 50-line windows (mean 41.0 LOC, 390.8 tokens) capture surrounding declarations, annotations, and helper calls, providing richer co-occurrence signals than isolated AST nodes (mean 26.7 LOC, 250.8 tokens). This token gap is expected from single-node extraction; without sibling merging, fine-grained AST slicing sacrifices contextual density. (2) *Contextual compensation*: headers inject architectural namespace, aiding cross-encoders to match requirement keywords against file and class names. Notably, AST headers carry more metadata (class and method cues), yet line windows remain superior. (3) *Query granularity*: composite JD queries (5–8 skills) may favor broader 50-LOC windows encompassing co-occurring functions over individual nodes.

B. Threats to Validity

Internal Validity: *Pooling and blinding.* Mitigated by pooling all 12 systems at top-5 (so top-5 is fully judged by construction) and dual annotation with annotators blinded to system, header condition, rank, and score ($\kappa_w = 0.9822$; dual author annotation may carry expectation bias). *Coverage and overlap.* Confounds from AST line coverage (69.0% lines; median 16 vs. 50 LOC) and window overlap (26.4% of Line top-10 slots involved in overlap [16.8% redundant], vs. 0% in AST No-Header and 0.8% in AST+Header due to nested units) were audited: common-support evaluation ($N = 826$ shared pairs) shows Line retains its lead ($\Delta = -0.111, p_{\mathrm{Holm}} = 0.0036$), and deduplication preserves ranking. *Truncation and unjudged bias.* The reranker truncates at 512 tokens (28.6% line vs. 15.8% AST chunks exceed this). Pooling depth (5) is smaller than evaluation depth (10); unjudged items at ranks 6–10 receive zero gain. Because Judged@10 varies across systems (0.560–0.752; 0.660–0.736 at reranking, lowest for AST+Header), this bias is not uniform, though bpref yields identical condition rankings. *Header asymmetry.* The two headers carry different metadata amounts (File and Lines for line windows; File, Class, Method, Lines for AST chunks), confounding the header effect with richness. *Parser configuration.* Grammars were extended before final pooling; no judgments were collected with earlier parsers. **Statistical Conclusion Validity:** With $N = 25$ queries, 80% power requires paired differences of at least $\Delta \approx 0.08$–0.13 (the minimum detectable effect, MDE), given observed standard deviations $\sigma_d \in [0.137, 0.184]$ and $\alpha \in [0.0125, 0.05]$; non-significant small differences (e.g., Comp 4, $\Delta = +0.041$) indicate absence of evidence rather than equivalence. **Construct Validity:** Requirements were extracted via `gemini-2.5-flash` (spot-checked by authors across the 25 JDs) to reflect authentic expectations. Curating core architectural modules yields cleaner candidate density than whole-repository search, though both strategies

TABLE III
RESEARCH ROADMAP AND MILESTONES

Phase	Objective	Target Data	Analysis Method
P1	Corpus Expansion (RQ3)	100 repos (Java, TS, Py, Go); 50 JDs/issues	Factorial; mixed-effects models
P2	Graph Headers (RQ3)	Call-graphs & imports vs. path headers	Paired Wilcoxon-Holm; ablations
P3	Human Study (RQ4)	$N = 20$ software developers	Accuracy, inspection time, confidence

share the same files. Relevance was judged on a 3-point ordinal graded-relevance scale with task-specific definitions (Section III-C). **External Validity:** Evaluated on Java Spring Boot and React/TypeScript with BGE-M3 and BGE-Reranker-Base; caution is warranted when generalizing to dynamically typed languages or distributed architectures.

VI. RESEARCH PLAN AND ROADMAP

Table III outlines planned phases for RQ3 and RQ4: corpus expansion across four languages (Java, TypeScript, Python, Go; **P1**), graph-enriched headers via precomputed SCIP/LSIF code-index formats (**P2**), and developer verification studies seeking ethics approval (**P3**).

VII. CONCLUSION

In this ERA paper, we presented a controlled 2×2 factorial evaluation crossing chunking strategy with context headers across 40 repositories. At reranking, line windows with headers achieved the highest NDCG@10 (0.621), outperforming AST chunks with headers ($\Delta = -0.134$; exploratory, unadjusted $p = 0.0003$). While context headers provide a substantial boost for AST chunks ($\Delta = +0.146$, $p_{\mathrm{Holm}} < 0.001$), line windows without headers significantly outperform both AST chunks with headers ($\Delta = -0.093$, $p_{\mathrm{Holm}} = 0.0438$) and AST chunks without headers ($\Delta = -0.239$, $p_{\mathrm{Holm}} < 0.001$). Together with coverage and overlap audits (0.0% fallback), this indicates that node-level AST extraction without sibling merging does not surpass sliding windows in requirement evidence retrieval. Our roadmap poses three open questions: (1) *Can graph-enriched headers bridge the information gap for fine-grained AST chunks?* (2) *How should code-RAG benchmarks weigh syntactic integrity against contextual density?* (3) *How can requirement retrieval best assist practitioners in verifying codebase capabilities?*

Data Availability: The replication package is available at: https://anonymous.4open.science/r/ast-rag-replication-2027-82C5/.

REFERENCES

[1] G. Antoniol, G. Canfora, G. Casazza, A. D. Lucia, and E. Merlo, "Recovering traceability links between code and documentation," *IEEE Transactions on Software Engineering*, vol. 28, no. 10, pp. 970–983, 2002.

[2] H. Gao, H. Kuang, W. K. G. Assunção, C. Mayr-Dorn, G. Rong, H. Zhang, X. Ma, and A. Egyed, "TRIAD: Automated traceability recovery based on biterm-enhanced deduction of transitive links among artifacts," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, 2024, pp. 2668–2680.

[3] Y. Zhang, X. Zhao, Z. Z. Wang, C. Yang, J. Wei, and T. Wu, "cAST: Enhancing code retrieval-augmented generation with structural chunking via abstract syntax tree," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025, pp. 8106–8116.

[4] X. Wu, J. Gong, G. Jahangirova, and J. M. Zhang, "How does chunking affect retrieval-augmented code completion? A controlled empirical study," *arXiv preprint arXiv:2605.04763*, 2026.

[5] T. Galimzyanov, O. Kolomyttseva, and E. Bogomolov, "Practical code RAG at scale: Task-aware retrieval design choices under compute budgets," *arXiv preprint arXiv:2510.20609*, 2025.

[6] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 1536–1547.

[7] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, "UniXcoder: Unified cross-modal pre-training for code representation," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, 2022, pp. 7212–7225.

[8] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "CodeSearchNet challenge: Evaluating the state of semantic code search," *arXiv preprint arXiv:1909.09436*, 2019.

[9] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE M3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," *arXiv preprint arXiv:2402.03216*, 2024.

[10] F. Hu, Y. Wang, L. Du, X. Li, H. Zhang, S. Han, and D. Zhang, "Revisiting code search in a two-stage paradigm," in *Proceedings of the 16th ACM International Conference on Web Search and Data Mining (WSDM)*, 2023, pp. 1066–1074.

[11] D. Zheng, Y. Wang, W. Chen, J. Chen, and Z. Zheng, "CoSTV: Accelerating code search with two-stage paradigm and vector retrieval," in *Proceedings of the 31st Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 2024.

[12] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J.-G. Lou, and W. Chen, "RepoCoder: Repository-level code completion through iterative retrieval and generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 2471–2484.

[13] T. Liu, C. Xu, and J. McAuley, "RepoBench: Benchmarking complete code generation with repository-level context," in *International Conference on Learning Representations (ICLR)*, 2024.

[14] Y. Ding, Z. Wang, W. U. Ahmad, H. Ding, M. Tan, N. Jain, M. K. Ramanathan, R. Nallapati, P. Bhatia, D. Roth, and B. Xiang, "CrossCodeEval: A diverse and multilingual benchmark for cross-file code completion," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[15] J. E. Montandon, L. L. Silva, and M. T. Valente, "Identifying experts in software libraries and frameworks among GitHub users," in *Proceedings of the 16th International Conference on Mining Software Repositories (MSR)*. IEEE, 2019, pp. 276–287.

[16] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[17] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, D. Lian, and J.-Y. Nie, "C-Pack: Packed resources for general Chinese embeddings," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2024.

[18] K. Järvelin and J. Kekäläinen, "Cumulated gain-based evaluation of IR techniques," *ACM Transactions on Information Systems*, vol. 20, no. 4, pp. 422–446, 2002.

[19] C. Buckley and E. M. Voorhees, "Retrieval evaluation with incomplete information," in *Proceedings of the 27th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 2004, pp. 25–32.

[20] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[21] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.